Bases de Data Engineering et Machine Learning

Séance 1 : Panorama du Data Engineering et du ML

Bac+4 Data, Institut GoldenCollar | Mouaad AGOURRAM

Faisons connaissance

Votre parcours	Python et Pandas	Vos attentes
Alternance, stage, entreprise : dans quel secteur ?	Jamais, un peu, ou au quotidien ?	Ce que vous voulez savoir faire en janvier

Ce que ce module ajoute au module BI

MODULE BI ET ETL

- Les principes de l'ETL
- La modélisation pour l'analyse
- Les tableaux de bord

CE MODULE

- Ingestion et contrôles automatisés en Python
- Pipelines reproductibles
- Préparation des données et premiers modèles prédictifs

Les objectifs du module

- Comprendre les rôles du Data Engineering et du Machine Learning dans un projet
- Automatiser l'ingestion et la validation des données avec Python et Pandas
- Construire un pipeline reproductible et préparer les variables pour le ML
- Mettre en œuvre un apprentissage supervisé et interpréter ses résultats
- Présenter et justifier ses choix techniques et ses résultats

Le module en 10 séances

Des données fiables

S1 22/10	Panorama du Data Engineering et du ML
S2 29/10	Ingestion automatisée avec Python
S3 05/11	Validation automatisée des données
S4 12/11	Pipelines Python reproductibles
S5 19/11	Préparer les données pour le ML

Des modèles prédictifs

S6 26/11	Introduction au ML supervisé
S7 03/12	Prétraitement pour le ML
S8 10/12	Premiers modèles prédictifs
S9 17/12	Évaluation des modèles
S10 07/01	Soutenances orales

Le projet du module

Données accessibles et documentées → **ETL automatisé et contrôlé** → **Modèle prédictif entraîné et validé**

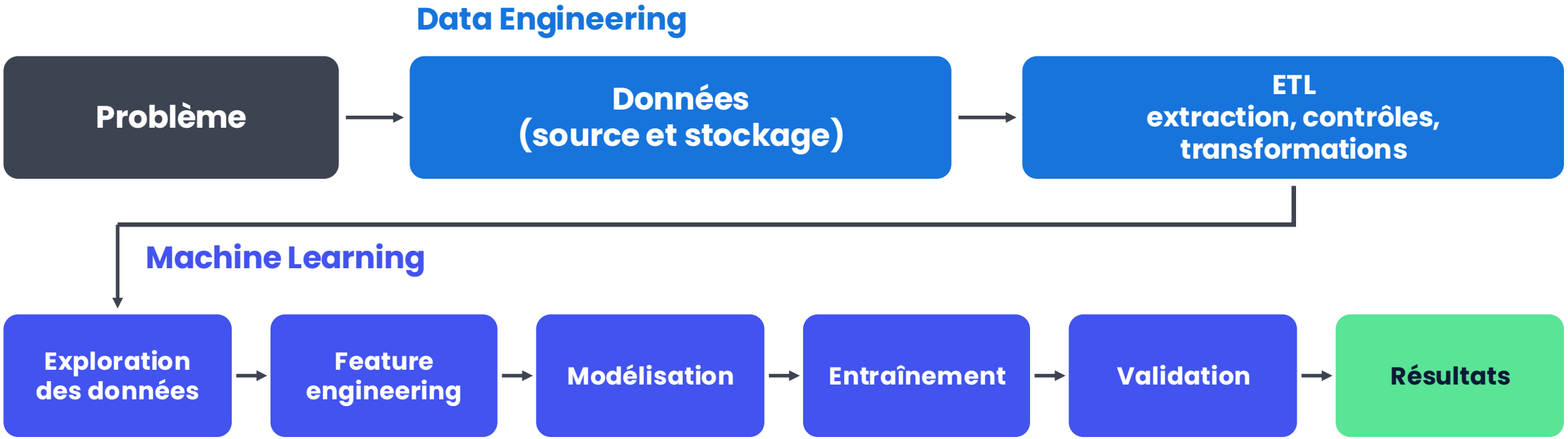
Trois livrables

- **Un court rapport**
- **Notebook et scripts reproductibles**
- **Support de soutenance**

Modalités d'évaluation

Critère	Proposition
Pertinence du problème, compréhension et exploration des données	20 %
Qualité de la préparation et reproductibilité du pipeline	30 %
Choix du modèle, évaluation et interprétation des résultats	30 %
Présentation orale, justification des choix et réponses aux questions	20 %

La soutenance



Séance 1

01 Pourquoi les données avant le modèle

02 Les rôles d'un projet ML

03 Le pipeline de données

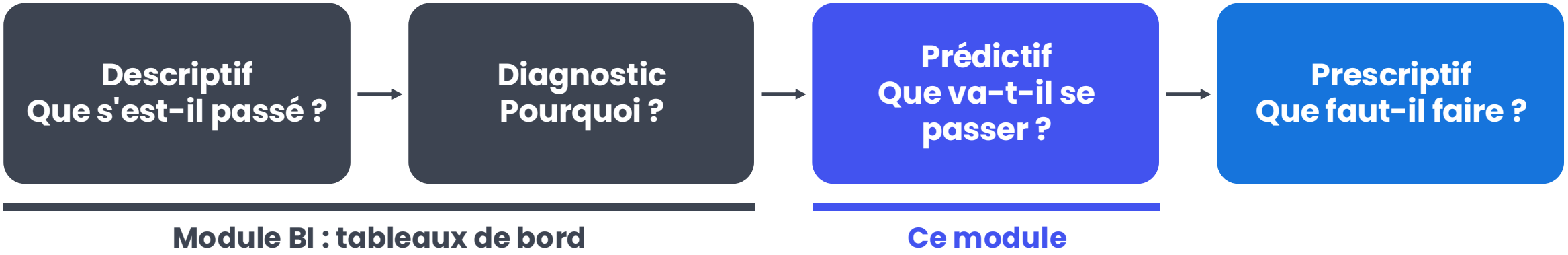
04 Le Machine Learning en bref

05 Démo : un pipeline ML avec ZenML

06 Poser les bonnes questions

Pourquoi les données avant le modèle

Du descriptif au prédictif



Prédire suppose un historique fiable

Un humain repère une valeur aberrante dans un tableau de bord. Un modèle, lui, l'apprend comme une vérité.

Où passe le temps en data science ?

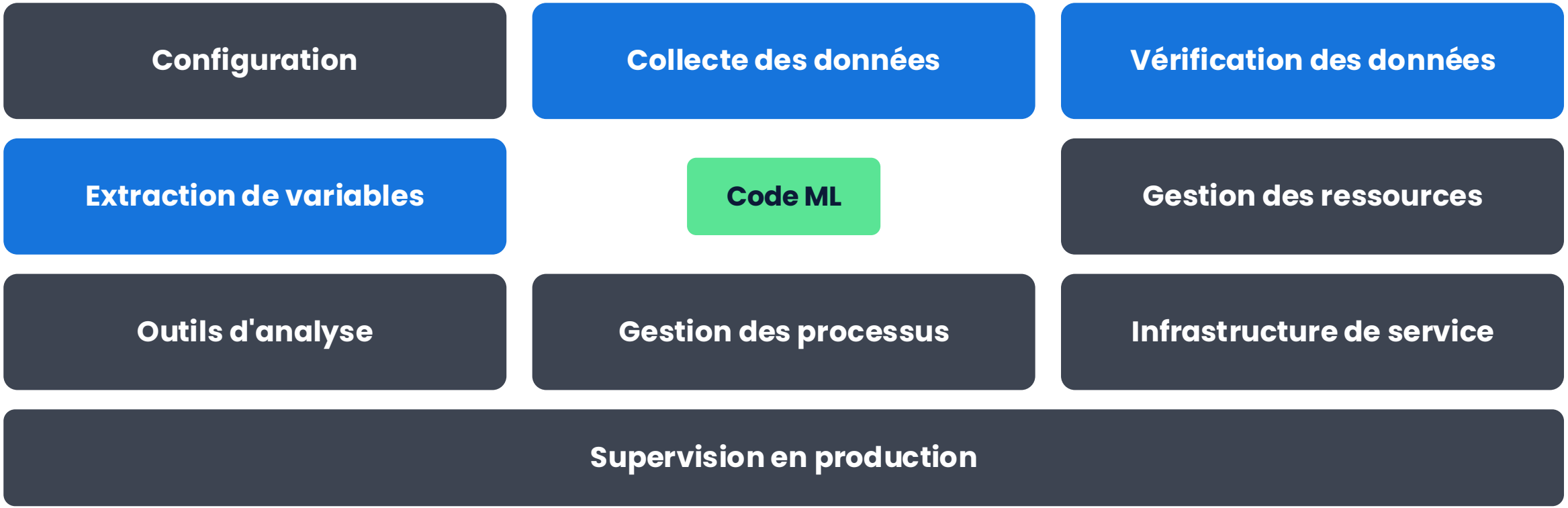
45 %

du temps passé à charger et à nettoyer les données



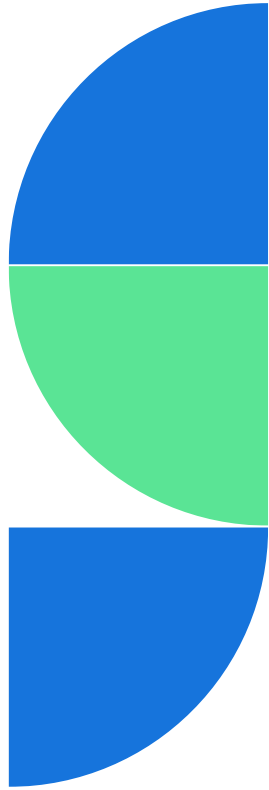
Sélection 11 % · entraînement et scoring 12 % · déploiement 11 % (n = 1 099).

Le modèle n'est qu'une petite pièce



En bleu : le cœur du Data Engineering, et de la première moitié du module.

Un modèle ne vaut pas mieux que les données qui le nourrissent.





Les rôles d'un projet ML

Qui fait quoi dans un projet ML ?

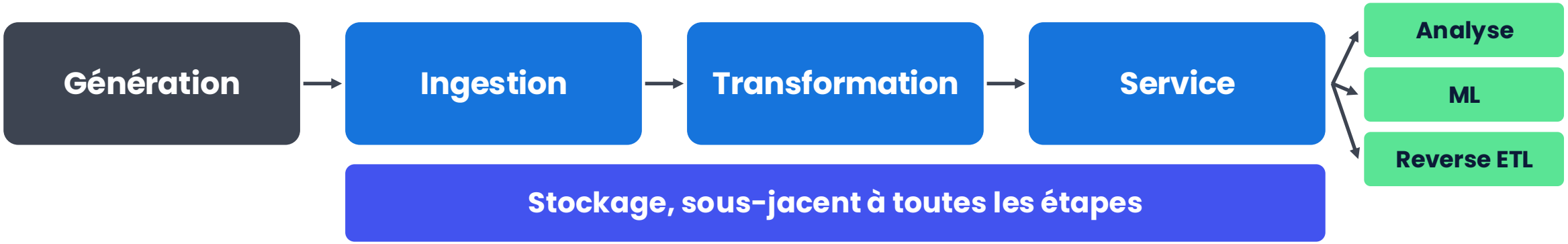
Rôle	Mission	Ce qu'il livre
Business Stakeholder	Fixe l'objectif métier et la mesure du succès	Le besoin, les critères
Solution Architect	Conçoit l'architecture, choisit les technologies	Le schéma d'architecture
Data Engineer	Rend les données disponibles et fiables	Pipelines, données de qualité
Data Scientist	Traduit le besoin en problème de ML, construit le modèle	Modèles, expériences
ML Engineer	Met en production, automatise, surveille	Pipelines ML, services
Data Analyst	Analyse le passé et l'explique	Analyses, tableaux de bord

Exercice 1 : à qui revient la tâche ?

Tâche	Rôle
1. Récupérer chaque nuit les ventes du site web	**Data Engineer**
2. Construire le tableau de bord des ventes mensuelles	**Data Analyst**
3. Bloquer un fichier reçu sans identifiant client	**Data Engineer**
4. Comparer deux modèles de prévision des ventes	**Data Scientist**
5. Exposer le modèle retenu derrière une API	**ML Engineer**
6. Expliquer au directeur commercial la baisse du trimestre	**Data Analyst**
7. Choisir les variables qui expliquent le départ d'un client	**Data Scientist**
8. Surveiller que les performances du modèle ne se dégradent pas	**ML Engineer**

Le pipeline de données

Le cycle de vie du Data Engineering



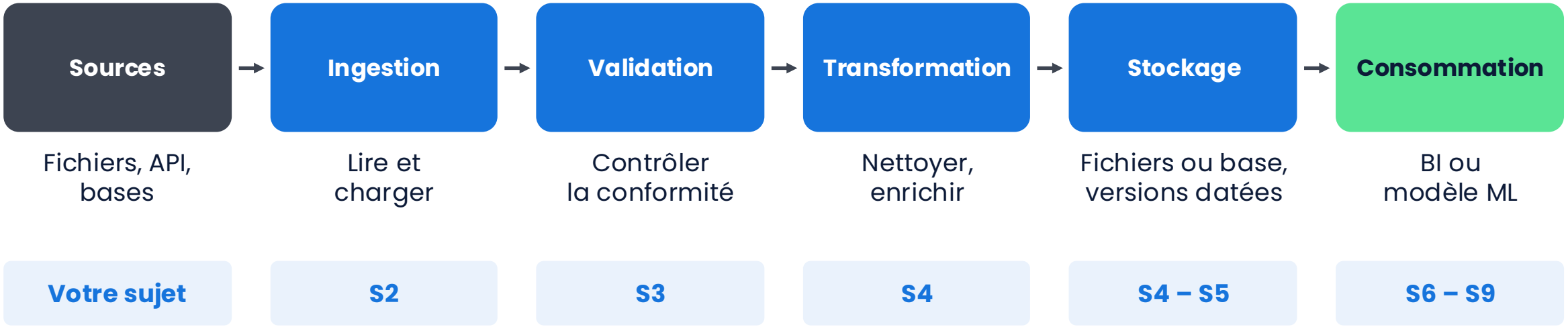
Les courants sous-jacents, présents à chaque étape

Sécurité	Gestion des données	DataOps	Architecture des données	Orchestration	Génie logiciel

Dans ce module

Ingestion, transformation et service en Python ; la qualité relève de la gestion des données.

Dans ce module : six étapes



Votre projet

Ce pipeline, exécuté de bout en bout sans intervention manuelle.

Chaque étape a son risque

Étape	Ce qui peut mal tourner	Le contrôle qui protège
Ingestion	Fichier absent, format changé, encodage	Présence, colonnes, nombre de lignes
Validation	Valeurs manquantes, doublons, valeurs absurdes	Règles de qualité, rapport d'anomalies
Transformation	Jointure qui duplique, unité mal convertie	Totaux comparés avant et après
Stockage	Version précédente écrasée	Données datées et versionnées
Consommation	Données préparées autrement qu'à l'entraînement	Mêmes transformations partout

La reproductibilité se prépare

EXÉCUTION IMPROVISÉE

- Cellules exécutées dans le désordre
- Dépendances et chemins propres à votre machine
- Aucune trace de ce qui a échoué

EXÉCUTION MAÎTRISÉE

- Des étapes exécutées dans un ordre fixé
- Dépendances déclarées, chemins configurables
- Logs, erreurs gérées, résultats vérifiables

Exercice 2 : la panne du lundi matin

Lundi, 9h

Le modèle annonce une chute de 40 % des ventes pour la région Nord. En magasin, les ventes semblent stables.

- Quelles questions posez-vous pour trouver la cause ?
- À quelle étape du pipeline le problème est-il entré ?
- Quel contrôle l'aurait arrêté ?

Réponse

Depuis vendredi, un magasin envoie ses montants en milliers d'euros. Entré à l'ingestion, aurait dû être arrêté à la validation : plage de valeurs attendue par magasin.

Le Machine Learning en bref

Programmer ou apprendre ?

Programmation classique



Apprentissage supervisé



Exemple : le spam

Classique : on écrit « contient GRATUIT → spam ».

ML : on montre des milliers d'e-mails déjà classés ; le modèle trouve les indices.

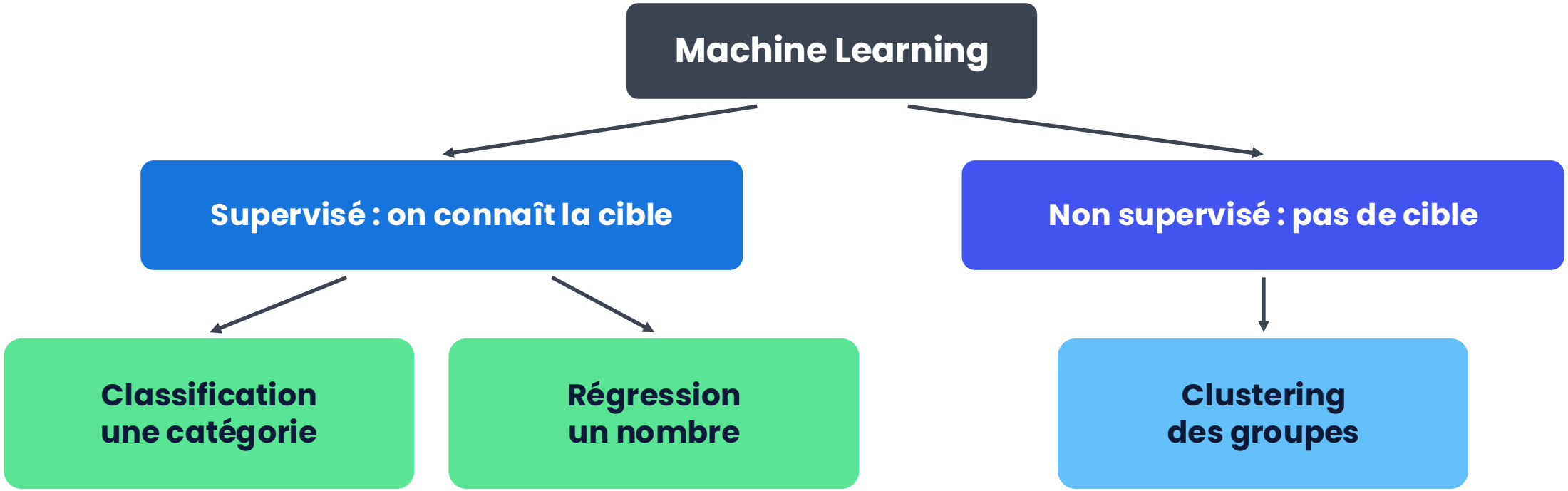
Une définition opérationnelle

> **Tom Mitchell, 1997**
> « Un programme apprend de l'expérience E, pour une tâche T et une mesure de performance P, si sa performance sur T, mesurée par P, s'améliore avec E. »

T, la tâche	E, l'expérience	P, la performance
Classer un e-mail : spam ou non	Des e-mails déjà étiquetés	La part d'e-mails bien classés

Pas de E, pas d'apprentissage : fournir E, c'est le travail du Data Engineering.

Les grandes familles du ML



Ce module : l'apprentissage supervisé, séances 6 à 9

Le vocabulaire sur le Titanic

Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Survived
3	male	22	1	0	7.25	S	0
1	female	38	1	0	71.28	C	1
3	female	26	0	0	7.93	S	1
1	female	35	1	0	53.10	S	1
3	male	35	0	0	8.05	S	0
3	male	NaN	0	0	8.46	Q	0

1 ligne = 1 observation : un passager

Numériques : Age, SibSp, Parch, Fare

Catégorielles : Pclass, Sex, Embarked

Cible : Survived (0 / 1), donc classification

Valeur manquante : 177 âges sur 891

Entraîner, puis prédire

Le modèle apprend le lien variables → cible sur les passagers connus, puis prédit Survived pour de nouveaux passagers.

Entraînement, validation, test

Entraînement (60 %)	Validation (20 %)	Test (20 %)
Ajuster les paramètres du modèle	Comparer les modèles, régler les hyperparamètres	Estimer la performance finale, une seule fois

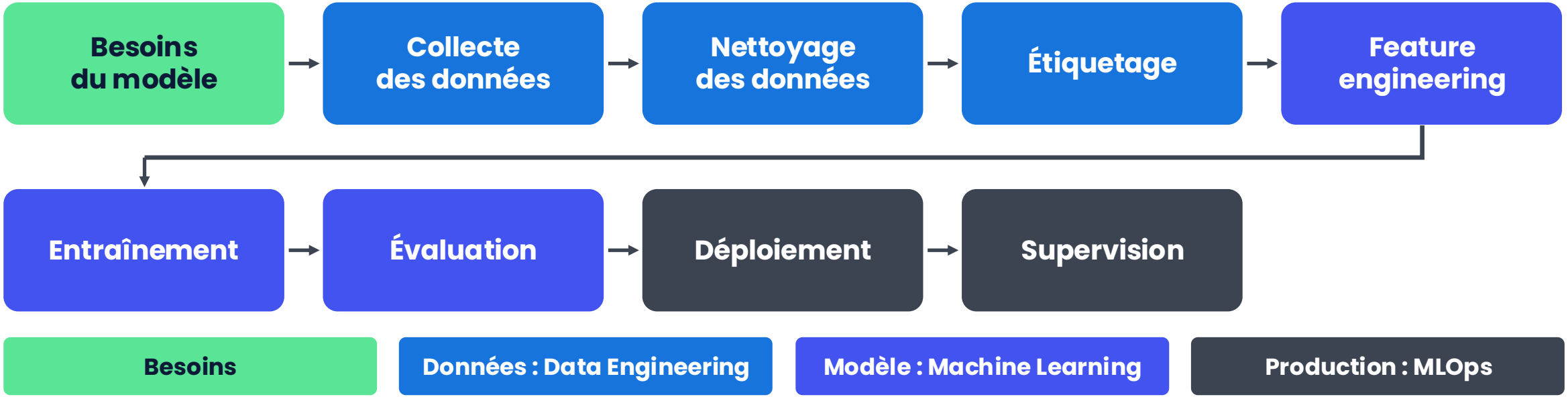
Peu de données : la validation croisée

L'entraînement est découpé en k blocs ; chaque bloc sert tour à tour de validation.

Le piège : la fuite d'information

Moyenne, médiane, encodage : tout s'apprend sur l'entraînement seulement, puis s'applique au reste.

Le workflow d'un projet ML



Deux constats de l'étude

Le workflow reboucle sans cesse. Et trouver, gérer et versionner les données est plus difficile que tout le reste.

Exercice 3 : quel type de problème ?

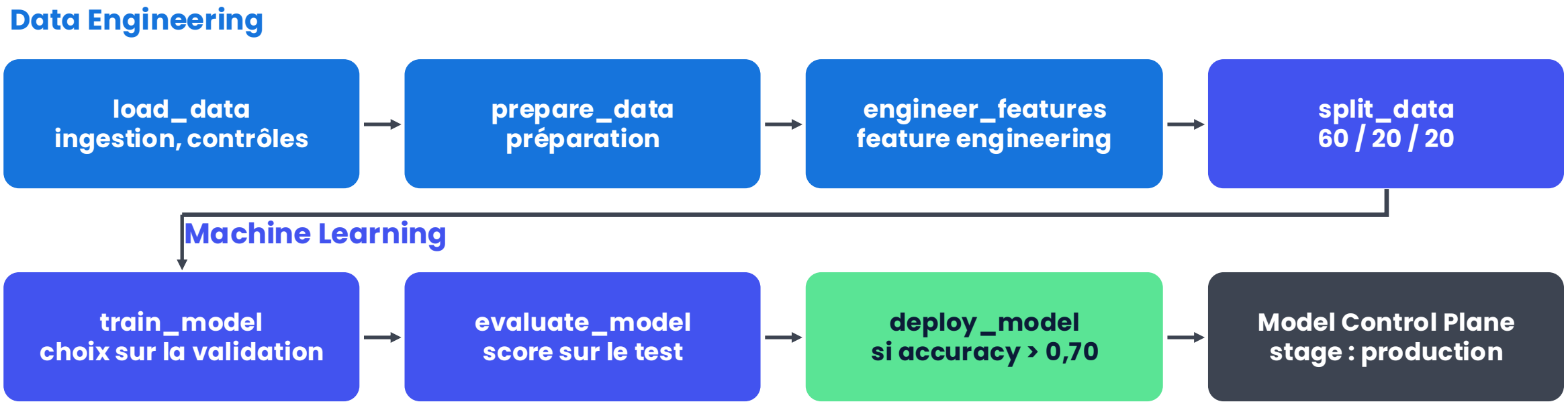
Situation	Type
1. Prévoir le nombre de vélos loués demain dans une station	**Régression**
2. Repérer une transaction bancaire frauduleuse	**Classification**
3. Calculer le chiffre d'affaires du mois dernier	**Pas du ML : une requête**
4. Estimer le prix de vente d'un appartement	**Régression**
5. Prédire si un abonné va résilier son abonnement	**Classification**
6. Trier les clients par ordre alphabétique	**Pas du ML : un tri**
7. Regrouper des clients aux comportements proches	**Non supervisé**

Démo : un pipeline ML avec ZenML

ZenML en cinq notions

Notion	Ce que c'est
Step	Une fonction Python décorée par @step
Pipeline	Une fonction @pipeline qui enchaîne les étapes : un graphe d'exécution
Artefact	Chaque sortie d'étape, enregistrée et versionnée ; base du cache
Stack	Où le pipeline s'exécute : orchestrateur et stockage des artefacts
Model Control Plane	Les versions d'un modèle, leurs métriques et leur stage (production)

Le pipeline de la démo



Les données : Titanic (Kaggle)

891 passagers, 342 survivants (38,4 %) ; 177 âges et 687 cabines manquants.

Étape 1 : ingestion et contrôles

```
@step(enable_cache=False)
def load_data(path):
    df = pd.read_csv(path)
    missing = EXPECTED - set(df.columns)
    if missing:
        raise ValueError(f"Colonnes manquantes : {missing}")
    if (df["Fare"] < 0).any():
        raise ValueError("Tarif négatif")
    return df
```

Rejeter tôt

Ces deux contrôles bloquent certaines anomalies. Les types et les autres valeurs demandent aussi des règles explicites.

Étape 2 : préparation

```
@step
def prepare_data(df):
    df = df.drop(columns=["Cabin", "Ticket"],
                 errors="ignore").copy()
    df["Embarked"] = df["Embarked"].str.strip().str.upper()
    return df  # PassengerId conservé, une ligne par entrée
```

Pourquoi supprimer Cabin ?

687 valeurs manquantes sur 891 (77 %). Les 177 âges manquants, eux, seront imputés plus tard, sur l'entraînement seulement.

Étape 3 : feature engineering

```
title = df["Name"].str.extract(r",\s*([^.]+)\.",
                               expand=False)
df["Title"] = title.where(title.isin(TITLES), "Rare")
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)
df = df.drop(columns=["Name", "SibSp", "Parch"])
```

Des variables construites, sans fuite

Le titre résume sexe, âge et statut ; FamilySize combine SibSp et Parch. Elles utilisent des informations disponibles avant la prédiction, sans statistique calculée sur les autres passagers.

Étape 4 : découpage 60 / 20 / 20

```
train, rest = train_test_split(
    df, test_size=0.4,
    stratify=df[TARGET], random_state=42)
validation, test = train_test_split(
    rest, test_size=0.5,
    stratify=rest[TARGET], random_state=42)
```

stratify et random_state

Proportions de survivants proches de 38 % dans les trois jeux ; un découpage fixé, donc une exécution reproductible.

Étape 5 : prétraitement sans fuite

```
numeric = make_pipeline(
    SimpleImputer(strategy="median"), StandardScaler())
categorical = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore"))
prep = ColumnTransformer([
    ("num", numeric, NUMERIC),
    ("cat", categorical, CATEGORICAL)])
```

Dans un Pipeline scikit-learn

La médiane des âges et l'encodage sont appris pendant fit, donc sur l'entraînement seulement, puis appliqués à la validation et au test.

Étape 6 : entraînement et validation

```
models, scores = {}, {}
for name, classifier in candidates.items():
    model = make_pipeline(make_preprocessor(), classifier)
    model.fit(*split_xy(train))
    scores[name] = model.score(*split_xy(validation))
    models[name] = model
best = max(scores, key=scores.get)
model = models[best]
```

Résultat de la démo

Validation : régression logistique 0,848, forêt aléatoire 0,843. La régression logistique est retenue.

Étape 7 : évaluer, puis promouvoir

```
accuracy = model.score(*split_xy(test))

@step(enable_cache=False)
def deploy_model(accuracy, threshold):
    if accuracy <= threshold:
        return False
    version = get_step_context().model
    version.set_stage("production", force=True)
    return True
```

Résultat de la démo

Test : 0,849 > 0,70, la version passe en production. Repère : prédire « décédé » pour tous donne 61,6 %.

Le pipeline d'entraînement

```
@pipeline(model=Model(name=MODEL_NAME))
def training_pipeline(path, threshold=0.70):
    df = load_data(path)
    df = prepare_data(df)
    df = engineer_features(df)
    train, validation, test = split_data(df)
    model = train_model(train, validation)
    accuracy = evaluate_model(model, test)
    deploy_model(accuracy, threshold)
```

Ce que ZenML enregistre

Artefacts et métriques traçables. Relecture du CSV et promotion hors cache, transformations réutilisables.

Inférence avec le modèle en production

```
model = get_pipeline_context().model
model = model.get_model_artifact("model")
df = engineer_features(prepare_data(load_data(path)))
predict(model, df)
```

Passager (fictif)	Classe	Sexe	Âge	Probabilité de survie
Jack Dawson	3	homme	20	0,11
Rose DeWitt Bukater	1	femme	17	0,95
Claire Martin	2	femme	34	0,76
Louis Petit	3	garçon	6	0,72

Lancer la démo

```
# Depuis seance1/demo_zenml, environnement activé
pip install -r requirements.txt
export OBJC_DISABLE_INITIALIZE_FORK_SAFETY=YES  # Mac
zenml login --local
zenml init
python run.py train
python run.py predict
zenml model version list --model titanic_survival
```

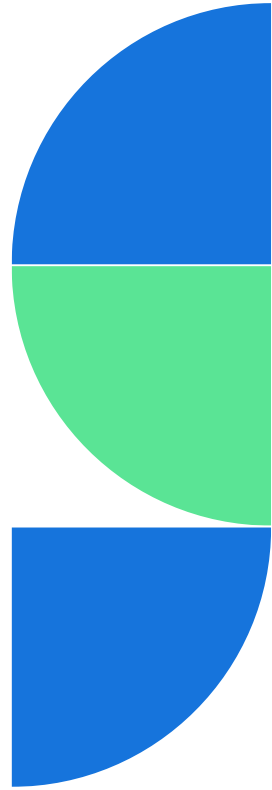
Dashboard local et commande CLI

Dashboard local : graphe et artefacts. CLI : versions du modèle et stages. La vue des modèles est réservée à Pro.

Poser les bonnes questions

Une architecture est une réponse.

Encore faut-il connaître la question.



Même demande, deux contextes

« On veut prédire les clients qui vont résilier leur abonnement. »

	Entreprise A	Entreprise B
D'où viennent les données ?	Un export CSV mensuel	Des événements en continu
Quand sert la prédiction ?	Le 1er du mois, campagne d'appels	Pendant l'appel, en < 1 s
→ **Ingestion**	Script planifié chaque mois	Lecture continue d'un flux
→ **Calcul des prédictions**	En lot, pour tous les clients	À la demande, client par client
→ **Mise à disposition**	Fichier Excel	API appelée par l'écran du conseiller

La grille des questions

1. Le métier

Quelle décision éclaire la prédiction ?
Qui l'utilise, et comment ?
Que coûte une erreur ?

2. Les données

Quelles sources, quels formats ?
Qui en est propriétaire ?
La cible existe-t-elle ?

3. Volume et rythme

Combien de lignes, de Go ?
À quelle fréquence ?
Cela va-t-il croître ?

4. La qualité

Valeurs manquantes, doublons ?
Des définitions partagées ?
Qui corrige à la source ?

5. L'usage

En direct, chaque jour,
chaque mois ?
Fichier, tableau ou API ?

6. Les contraintes

Données personnelles, RGPD ?
Budget, outils en place ?
Qui maintiendra ?

De la réponse à l'architecture

Si on vous répond…	… l'architecture prévoit
Un export CSV déposé chaque nuit	Un script d'ingestion planifié chaque nuit
Une API qui renvoie 100 lignes par page	Une ingestion paginée, avec reprise sur erreur
Quelques centaines de Mo	Pandas sur une seule machine
Plusieurs centaines de Go par jour	Traitement par morceaux ou distribué, selon les besoins
Des définitions différentes selon les services	Un dictionnaire de données et des règles de validation
Une liste utilisée une fois par mois	Des prédictions en lot, pas d'API temps réel

Questions-besoins, questions-solutions

À ÉVITER EN PREMIER

- « On utilise quel outil ? »
- « On fait du deep learning ? »
- « On met tout dans le cloud ? »

À POSER D'ABORD

- « Quelle décision voulez-vous prendre ? »
- « À quel moment avez-vous besoin de la réponse ? »
- « Comment savez-vous aujourd'hui qu'un client est parti ? »

Votre livrable : un schéma argumenté



Hypothèses	Justifications	Questions ouvertes
Ce que vous supposez, faute de réponse	Chaque choix renvoie à une réponse du client	Ce qu'il reste à demander

Atelier : la demande du client

FitCity, 12 salles de sport (entreprise fictive)

« Nous perdons trop d'abonnés. On aimerait utiliser l'IA pour savoir qui va partir. Vous pouvez nous proposer quelque chose ? »

- 5 min : classer cinq questions, justifier les deux premières
- 5 min : lire l'email, relever trois faits et une question oubliée
- 15 min : architecture, deux choix justifiés et une réaction à une anomalie
- 6 min : deux restitutions, puis 4 min de discussion

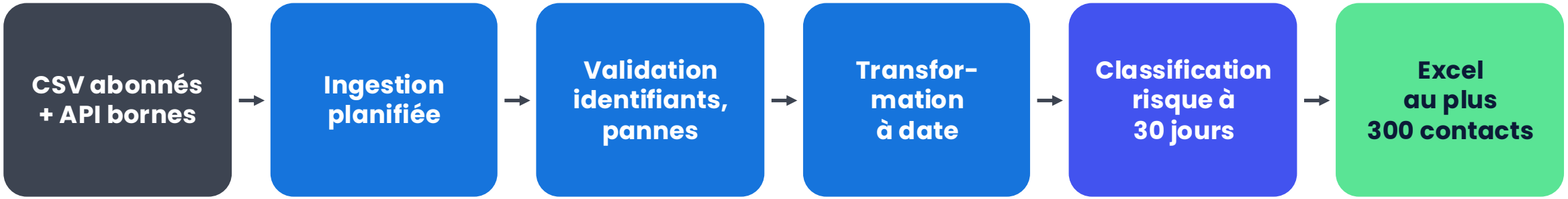
Atelier

La demande de FitCity
Fiche étudiants, puis email du client
Durée : 35 min

Correction : les réponses du client

Famille	Réponse clé
Métier	Au plus 300 contacts mensuels, quatre conseillers sur Excel
Données	CSV : type_fin, date_effet, date_notification ; badges JSON ; archives depuis 2021
Volume	18 000 actifs, quelques Go par an
Qualité	Numéros locaux à rapprocher ; panne de borne différente de zéro visite
Usage	Score au 1er : départ définitif à 30 jours, hors suspensions et départs annoncés
Contraintes	Motif médical exclu ; petit budget et maintenance simple

Correction : une architecture possible



Chaque choix renvoie à une réponse

Population au 1er : actifs, hors suspensions et départs déjà annoncés.
Variables connues à la date du score. Test sur des mois ultérieurs.
Stockage : fichiers datés ou base. Panne de borne différente de zéro visite.
Attendu : prioriser 300 contacts. Calcul de métrique en prolongement.

Cas d'usage

Quatre cas d'usage classiques

Cas	Cible	Type	La question décisive
Départ client (télécoms)	Résilie : oui / non	Classification	Combien de temps à l'avance faut-il prévenir ?
Fraude à la carte	Fraude : oui / non	Classification	Quelle part des transactions est frauduleuse ?
Prévision de la demande	Ventes de la semaine	Régression	À quel niveau : magasin, produit, jour ?
Prix immobilier	Prix de vente	Régression	Quelles transactions, sur quelle période ?

Fraude : la question qui change tout

0,17 %

des transactions sont des fraudes
492 sur 284 807

Le piège
Un modèle qui répond toujours « pas de fraude » a 99,83 % de bonnes réponses… et ne détecte rien.

La question à poser
Quelle est la proportion de chaque classe ? Elle décide de la métrique (séance 9).

Lancement du projet

Les critères d'un bon sujet

Une cible claire

Une colonne à prédire, présente dans l'historique.

Des données accessibles

Téléchargeables, réutilisables légalement, sans données sensibles.

Une taille raisonnable

Un volume adapté à la mémoire, aux variables et au temps disponible.

Une décision derrière

Quelqu'un agirait différemment grâce à la prédiction.

Où trouver des données

Source	Ce qu'on y trouve	Exemple
data.gouv.fr	Les données publiques françaises	DVF : les ventes immobilières
INSEE	Statistiques démographiques et économiques	Revenus par commune
Kaggle Datasets	Des jeux variés, déjà documentés	Telco Customer Churn
UCI ML Repository	Des jeux classiques pour le ML	Wine Quality
OpenML	Des jeux avec des tâches de référence	Bank Marketing

Quatre pièges à éviter

- Aucun contrôle automatique, même sur un jeu propre
- La cible absente : on n'apprend pas ce qu'on n'a jamais observé
- Le volume démesuré : 50 Go sur un ordinateur portable
- Les données personnelles : noms, e-mails, données de santé

Avant la séance 2

- Choisir votre organisation
- Choisir un problème et un jeu de données
- Appliquer la grille des questions à votre sujet : une page
- Ouvrir vos données avec Pandas

Séance 2 : ingestion

Lire des CSV et des JSON
Contrôler l'import
Écrire des scripts réutilisables

Venir avec Python et Pandas installés

Synthèse

Ce qu'il faut retenir

Les données d'abord

Un modèle ne vaut pas mieux que le pipeline qui le nourrit.

Entraîner, valider, tester

Trois jeux distincts ; le test ne sert qu'une fois.

Les questions d'abord

Ce sont les réponses du métier qui décident de l'architecture.

Quiz de la séance 1

Question 1

Quelle question relève de l'analyse prédictive ?

A. Combien a-t-on vendu en mars ?

B. Pourquoi les ventes ont-elles baissé ?

C. Combien vendra-t-on le mois prochain ?

D. Quel magasin vend le plus ?

Question 2

Quel rôle joue Survived dans cette démo ?

A. Une variable explicative

B. Une valeur manquante

C. La cible d'une classification

D. Un identifiant

Question 3

À quoi sert le jeu de validation ?

A. À estimer la performance finale

B. À comparer les modèles et régler les hyperparamètres

C. À entraîner le modèle

D. À remplacer les valeurs manquantes

Question 4

Pourquoi apprendre la médiane sur l'entraînement ?

A. Pour aller plus vite

B. Pour garder le test hors de l'apprentissage

C. Parce que Pandas l'exige

D. Pour avoir plus de données

Question 5

Dans la démo, que fait deploy_model si l'accuracy vaut 0,65 ?

A. Il passe le modèle en production

B. Il ne déploie pas cette version

C. Il réentraîne le modèle

D. Il abaisse le seuil

Question 6

Un client demande « un modèle d'IA ». Quelle question poser en premier ?

A. Quel outil utilisez-vous ?

B. Voulez-vous du deep learning ?

C. Quelle décision la prédiction doit-elle éclairer ?

D. Combien de serveurs avez-vous ?

Merci ! Des questions ?

